Aegis Protocol — V0.1 Family Prototype

Technical Design Document

Version: 1.3.0 (Engineering Spec)
Scope: Sandbox-only, testnet-only, restricted to immediate family
Status: Ready for implementation

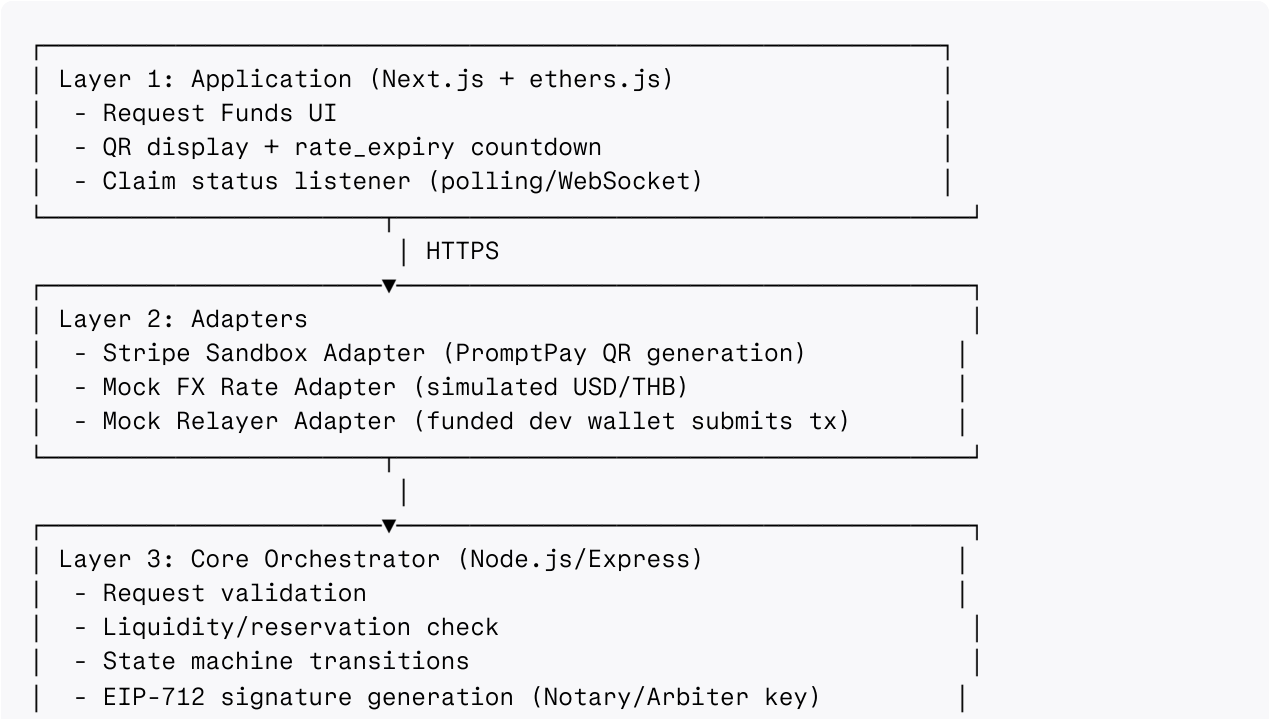
1. Purpose & Scope

V0.1 validates the core settlement mechanics of Aegis Protocol — fiat collection, exchange-rate locking, cryptographic notarization, and gasless stablecoin release — entirely in sandbox/testnet environments, with no real money, no real users outside your family, and no external compliance exposure.

Explicitly out of scope for V0.1:

- Mainnet deployment
- Real fiat movement
- Non-family users
- Automated liquidity replenishment
- Any Triple-A / production fiat gateway integration

2. System Architecture



```
└──────────────────────────┬──────────────────────────────────────────┘
                           │
┌──────────────────────────▼──────────────────────────────────────────┐
│ Layer 4: Settlement                                                 │
│  - PostgreSQL: transactions, processed_webhooks, reservations       │
│  - CoreEscrow.sol on Polygon Amoy (testnet USDC)                    │
└─────────────────────────────────────────────────────────────────────┘
```

3. Database Schema

```
CREATE TABLE transactions (
    tx_id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    beneficiary_wallet TEXT NOT NULL,
    requested_thb   NUMERIC(12,2) NOT NULL,
    locked_rate     NUMERIC(12,6) NOT NULL,
    usdc_amount     NUMERIC(18,6) NOT NULL,
    fee_amount      NUMERIC(18,6) NOT NULL DEFAULT 0,
    reserved_amount NUMERIC(18,6),
    reservation_expiry TIMESTAMPTZ,
    rate_expiry     TIMESTAMPTZ NOT NULL,
    status          TEXT NOT NULL DEFAULT 'pending_fiat'
                    CHECK (status IN (
                        'pending_fiat','fiat_confirmed','signature_issued',
                        'claim_failed','claimed','expired','failed'
                    )),
    signature       TEXT,
    nonce_or_txid   TEXT NOT NULL UNIQUE,
    stripe_payment_intent TEXT,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE processed_webhooks (
    event_id        TEXT PRIMARY KEY,
    processed_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE liquidity_pool (
    id              INT PRIMARY KEY DEFAULT 1,
    total_escrowed  NUMERIC(18,6) NOT NULL,
    total_reserved  NUMERIC(18,6) NOT NULL DEFAULT 0,
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

4. Smart Contract Spec — `CoreEscrow.sol`

Imports: OpenZeppelin `ReentrancyGuard`, `EIP712`, `ECDSA`, `IERC20`

State:

- `mapping(bytes32 => bool) usedTxIds` — replay protection, keyed by `txId`

- `mapping(address => uint256) depositBalances` — depositor accounting for refunds
- `address public arbiter` — the Orchestrator's notary address
- `IERC20 public usdc`

EIP-712 signed struct:

```
struct ReleaseRequest {
    bytes32 txId;
    address beneficiary;
    uint256 amount;
    uint256 deadline;
}
```

Functions:

Function	Behavior
`depositFunds(uint256 amount)`	Depositor (you) transfers USDC into escrow; increments `depositBalances` and pool total
`releaseFunds(ReleaseRequest req, bytes signature)`	Verifies signature from `arbiter`, checks `!usedTxIds[req.txId]`, checks `block.timestamp <= req.deadline`, marks `usedTxIds[req.txId] = true`, transfers `req.amount` USDC to `req.beneficiary`. Callable by **any address** (relayer support)
`cancelTransaction(bytes32 txId, uint256 amount)`	Allows depositor to reclaim funds if a timeout has passed and `txId` was never used

Security requirements:

- Mark `usedTxIds[req.txId] = true` **before** the external token transfer (checks-effects-interactions pattern)
- `nonReentrant` modifier on `releaseFunds` and `cancelTransaction`
- Domain separator must hardcode `chainId` and `verifyingContract` (Polygon Amoy chain ID: 80002)

5. Core Orchestrator — API Endpoints

`POST /request-funds`

1. Validate beneficiary wallet address format.
2. Query `liquidity_pool`: check `total_escrowed - total_reserved >= requested_usdc_equivalent`. Reject with `503 insufficient_liquidity` if not.
3. Fetch mock USD/THB rate.
4. Compute `usdc_amount = requested_thb / rate`, apply `fee_amount`.
5. Insert `transactions` row: status `pending_fiat`, `locked_rate`, `rate_expiry = now() + 15min`.

6. **Reserve liquidity**: increment `liquidity_pool.total_reserved` by `usdc_amount`; set `reservation_expiry = rate_expiry`.
7. Call Stripe Sandbox API → generate PromptPay QR for `requested_thb`.
8. Return `{ tx_id, qr_code_url, rate_expiry }`.

`POST /webhook/stripe`

1. Verify Stripe signature (5-minute tolerance window).
2. Extract `event_id`. `INSERT INTO processed_webhooks (event_id) ... ON CONFLICT DO NOTHING`. If conflict → return `200 OK` immediately, halt.
3. Look up `tx_id` from `payment_intent` metadata.
4. Assert `status = 'pending_fiat'` AND `now() < rate_expiry`. If rate expired → mark `failed`, release reservation, return `200 OK`.
5. Update status → `fiat_confirmed`.
6. Generate EIP-712 signature: `{ txId: tx_id, beneficiary, amount: usdc_amount, deadline: now() + 1hr }`, signed by Arbiter key (env var for V0.1; KMS-simulated).
7. Store signature, update status → `signature_issued`.
8. Convert reservation → real escrow debit (decrement `total_escrowed` and `total_reserved` together once claimed — see below).
9. Return `200 OK`.

`GET /transaction/:tx_id/status`

Polled by frontend. Returns current status, signature (if issued), and countdown fields.

Background job: `retry-stuck-claims` (runs every 60s)

- Find transactions in `signature_issued` for > 2 minutes.
- Re-invoke mock relayer submission with the same stored signature (safe — `txId` is idempotent on-chain).
- On confirmed on-chain receipt → status `claimed`, finalize liquidity pool debit.
- On repeated failure (3 attempts) → status `claim_failed`, alert for manual review.

6. State Machine

```
pending_fiat ──(webhook + rate valid)──▶ fiat_confirmed
fiat_confirmed ──(signature generated)──▶ signature_issued
signature_issued ──(relayer tx confirmed)──▶ claimed
signature_issued ──(relayer tx fails ×3)──▶ claim_failed
pending_fiat ──(rate_expiry passed, no webhook)──▶ expired
pending_fiat/fiat_confirmed ──(any unrecoverable error)──▶ failed
```

All transitions are single UPDATE statements guarded by a `WHERE status = '<expected_prior_state>'` clause to prevent race conditions from concurrent workers.

7. Relayer Model (V0.1 Simulation)

- A single funded developer wallet on Polygon Amoy acts as the "Relayer."
- Backend route `/internal/relay-submit` takes `{ txId, beneficiary, amount, deadline, signature }` and calls `releaseFunds` on-chain, paying testnet gas itself.
- This simulates the production Gelato/Biconomy relayer pattern without integrating a real relayer network yet.

8. Liquidity Model (V0.1)

- **Model B (prefunded)**, manually funded: you call `depositFunds` with testnet USDC to seed the escrow before running any family test.
- Reservation pattern prevents two concurrent requests from over-committing the same float (see `liquidity_pool.total_reserved`).
- No auto-replenishment in V0.1 — you top up manually via `depositFunds` when the pool runs low.

9. CLI Agent Build Prompts (Sequential)

1. **Contract:** Solidity `CoreEscrow.sol` per Section 4 — include `depositFunds`, `releaseFunds`, `cancelTransaction`, `usedTxIds` mapping, `nonReentrant`, EIP-712 domain with hardcoded chainId/verifyingContract.
2. **API Adapter:** Express `/request-funds` endpoint per Section 5 — including liquidity check and reservation logic, not just QR generation.
3. **Webhook Handler:** Express `/webhook/stripe` per Section 5 — idempotency via `processed_webhooks`, atomic state transition, EIP-712 signing via ethers.js.
4. **Frontend:** Next.js UI — Request Funds button, QR + countdown display, status polling, Claim triggers `/internal/relay-submit` (not direct user wallet gas payment).
5. **Reconciliation Job:** Node.js cron/background worker implementing `retry-stuck-claims` per Section 5.

10. Test Plan Before Calling V0.1 "Done"

- Single happy-path transaction end to end.
- Duplicate Stripe webhook delivery (simulate retry) → confirm only one signature issued.
- Two concurrent `/request-funds` calls exceeding combined pool balance → confirm second is rejected, not double-spent.

- Signature replay attempt (resubmit same signature twice) → confirm second call reverts on-chain.
- Rate expiry during pending fiat window → confirm transaction marked `expired`, reservation released.
- Relayer submission failure (simulate by pausing the relay route) → confirm `claim_failed` status and retry job behavior.